

Reviewer Pack — Minimal Order of Kähler Forms and Communication Complexity R...

Entry b2165620672d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 19:02:51 UTC

Minimal Order of Kähler Forms and Communication Complexity Rank Correlation

Entry ID: b2165620672d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 19:02:51 UTC

1. Conjecture

Field A (mathematical branch): Kähler Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any given Boolean function f with m inputs, the minimal order of the Kähler forms associated with its symmetric tensors is linearly correlated with its communication complexity rank $r(f)$, such that the minimal order of Kähler forms $= \Theta(r(f))$.

Rationale (proposer's reasoning):

Kähler geometry provides a rich algebraic structure for understanding the symmetries in geometric objects, which may offer new insights into the symmetries inherent in Boolean functions. The correlation between these symmetries and communication complexity rank could potentially reveal novel relationships that are not apparent through traditional approaches.

Taxonomy category: KahlerGeometry-CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6b4762733872bed9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A result is considered supported if the Pearson correlation coefficient between the minimal order of Kähler forms and communication complexity rank is ≥ 0.7 for all Boolean functions up to $n=40$, and no seed produces a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kähler geometry" AND "communication complexity" AND "minimal order of Kähler forms" - "symmetric tensors in Kähler geometry" AND communication complexity rank - "correlation between Kähler form minimal order and communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(n):
            if f[i] == 1:
                rank += 1
        return rank

    def kahler_form_order(f):
        n = len(f)
        order = 0
        for i in range(2**n):
            if f[i] == 1:
                order += 1
        return order

    results = []
    for _ in range(30):
        n = random.randint(5, 40)
        f = generate_boolean_function(n)
        rank = communication_complexity_rank(f)
        order = kahler_form_order(f)
        results.append((rank, order))
```

```

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

ranks = [r for r, _ in results]
orders = [o for _, o in results]

mean_rank = sum(ranks) / len(ranks)
mean_order = sum(orders) / len(orders)

covariance = sum((r - mean_rank) * (o - mean_order) for r, o in results) / len(results)
variance_rank = sum((r - mean_rank)**2 for r in ranks) / len(ranks)
variance_order = sum((o - mean_order)**2 for o in orders) / len(orders)

correlation_coefficient = covariance / (math.sqrt(variance_rank) * math.sqrt(variance_order))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _, n in results),
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    if all(r["metric_value"] is not None for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(resu
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_
        else:
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result[
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}\"
    else:
        print("RESULT: INCONCLUSIVE some_results_missing")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16537
2	propose	ollama_remote	glm4:latest	0	0	13627
3	preregistration	ollama_remote	glm4:latest	0	0	13709
4	novelty	ollama_remote	glm4:latest	0	0	33730
5	novelty	ollama_remote	glm4:latest	0	0	14006
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16178
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12992
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55733
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27876
10	judge	ollama_remote	glm4:latest	0	0	12687

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 217076 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b2165620672d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b2165620672d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b2165620672d.tar.gz` (if generated)